

Kevin Aimaier

📍 武汉 | ✉ kflyn825@outlook.com | ☎ 185 0162 1780 | 🌐 kevin-825

个人简历/求职意向 Summary

具有丰富芯片底层开发经验的嵌入式软件工程师，专注于 RISC-V 架构及其生态系统，熟练运用 C 语言进行高效、可靠的底层驱动和系统开发。具备扎实的计算机体系结构、操作系统原理和软件工程能力，能够快速适应并攻克复杂的技术难题。寻求与芯片设计、AI 加速器等前沿技术相关的挑战性岗位，期待在创新驱动的环境中持续成长并创造价值。

个人优势 & 专业技能 Core Competencies

多家芯片公司底层开发经验: 在多家芯片原厂（包括华为、君正等）的 Soc 开发软件团队中积累了丰富的芯片底层开发经验，对芯片内部模块比较熟悉(ADIE,BDIE)熟悉从硅前验证到硅后 Bring-up 的全流程，具备扎实的 SoC 架构理解和实际开发能力。

深入理解 RISC-V 及多种处理器架构: 熟练使用 C 语言进行底层开发，熟悉 Python、Bash Shell、Matlab/Simulink 等工具；精通 RISC-V、ARMv7/v8-M、AArch64 以及 Xtensa DSP 架构；具备丰富的芯片底层开发经验，熟悉 FPGA 验证和 Bring-up；深入理解 RTOS 和 Linux 内核，熟练使用 GNU 工具链和调试工具；善于运用 Generative AI 辅助编程和问题排查。

编程语言: C (8 年底层与驱动实战), Python, Bash Shell, Matlab/Simulink, JIM tcl, JSON/YAML, powershell.

处理器架构: 深入掌握 RISC-V, ARMv7/v8-M, AArch64 以及 Xtensa DSP 指令集架构；精通多种中断控制模型 (CLINT/PLIC/AIA, NVIC/GIC)、特权/异常模型 (Machine/User mode, EL0-EL3) 及内存保护单元 (PMP/MPU, MMU 与 Cache 一致性)。

芯片底层开发: 具备多家芯片原厂 SoC 开发经验，熟悉内部总线 (AHB/APB, Matrix/Arbiter)；熟悉 FPGA 硅前/硅后 Bring-up 验证；熟练开发 BootROM/Bootloader 及底层 HAL 驱动 (涵盖 I2C, SPI, UART, WDT, DMA, NOR/NAND, DDR 控制器等)。

操作系统内核: 具备丰富的 RTOS (FreeRTOS, LiteOS) 及 Linux 内核实战经验；深入理解内核任务调度 (CFS/FIFO/RR)、内存寻址与管理、同步原语及 VFS/sysfs 子系统；熟练掌握 Linux 字符设备驱动、设备树 (Device Tree)、构建系统 (Buildroot, Kconfig) 及系统初始化流程 (U-boot, initrd, systemd)。

调试与工具链: 精通 GNU 工具链 (GCC, GDB, nm, objdump) 及 CMake/Makefile 构建；具备丰富的底层 Debug 经验 (Trace, 内存踩踏/泄漏, Coredump, 宕机现场分析)；熟练应用 QEMU 模拟器、Docker 容器化开发以及 CI/CD (GitLab, Jenkins) 自动化流程。

综合与进阶: 熟练运用 Generative AI (Gemini Pro, Copilot) 进行辅助编程与复杂问题排查；具备极强的全英文原版技术文献解析与自主学习能力 (持续跟进 MIT OCW, Linux Foundation 等前沿硬核课程)。

工作经历 Experience

合肥君正科技有限公司 2025.01 - 2025.05

嵌入式软件工程师 | 合肥和武汉分公司

“项目 1：参与自研异构 AI MCU SoC 在 FPGA 上硅前验证，该芯片是 RISC-V 大小核架构 + TNPU，带 MMU 和 64KB Cache（64Byte 8-way i/d cache line），大核性能对标 arm coretex-A15 及以上，面向安防/学习机/扫地机器人/边缘 AI 等应用场景。”

- **提议集成 JTAG 并软硬联调：**在极度缺乏 FPGA 验证资源（8-10 人共用 2 台，周末无休）且无 JTAG 调试环境的情况下，提议并推进 JTAG 模块集成，并配合 SoC 部门耗时一月有余修复 GDB JTAG 断链不稳定问题以及 JTAG 子板问题，成功拉起首套稳定可用的 GDB OpenOCD JTAG 软硬联调环境，结束了只靠串口打印的“盲人摸象”式无效和 FPGA 拉线看波形的低效 debug 窘境，使得 debug 体验效率有质的提升。
- **PLIC 中断嵌套实现：**无 JTAG debug 手段的情况下，利用生成式 AI（ChatGPT, Copilot 和 Gemini Pro）完成了复杂的 QEMU 模拟器模拟 riscv CPU 和 PLIC 硬件，完成 CPU 与 PLIC 的软硬件协同仿真，在这个模拟环境下运行 FreeRTOS Demo 并完成 PLIC 嵌套中断功能开发，完成开发后 FPGA 上运行验证，成功排查出 SoC 部门提供的子系统中断号偏移的 Bug，使能 58 号中断结果 60 号中断触发 58 不触发。
- **Cache/Spinlock 底层适配优化：**研发 cache 驱动以支持 cache flush/invalidate 操作（按地址刷和按 cache set index 刷）。编写 spinlock 底层 riscv 汇编代码实现 spinlock 功能用以多核间自旋互斥操作。
- **检查修复 FreeRTOS 底层适配：**修复第一个任务初始化导致的栈踩踏 Bug 及临界区中断关闭恢复 Bug，协助解决自旋锁死锁。
- **静态编译 windows 版 OpenOCD 并生成容器化环境 DockerImage：**因 Windows 机器上开虚拟机 Linux 运行 OpenOCD 性能损失大难以满足 Ghz 处理器高速 jtag GDB debug 需求，大量运用 AI 编码辅助工具（Github Copilot 和 Gemini Pro）编写 Dockerfile 生成 open OCD Windows 版本编译环境 Image，用于后续编译更新的 OpenOCD 源码以支持 JTAG debug。OpenOCD 源码静态编译 Windows 版 OpenOCD 的挑战在于其依赖包非常的多且繁杂，大部分都需要手动编译源码形成静态库才能链接到 OpenOCD 中，并且编译 FLAG 需要针对每个依赖包进行细致调整编译 FLAG，最终成功生成了 Windows 版 OpenOCD 的 DockerImage。有了这个容器环境，使得新手编译 OpenOPCD 源码的时间从 2-3 天缩短到 5 分钟，极大提升了团队成员更新 OpenOCD 源码的积极性和效率。
- **GDB 初始化脚本编写以及调试修复严重 bug：**独立编写适合当前工程的 GDB init 脚本（主要包含自动加载可执行 elf 文件并自动连接目标板并下载，还有一些项目实用的 CMD）极大缩短了调试前期工作开销。利用写好的 GDB init 脚本进行实战调试并调试，反汇编内存现场分析，成功定位困扰算法部门很久的紧急 Bug（TNPU 越界写坏大核中断向量表）并且配合算法部分同事一步一步定位并成功找到触发异常的代码。
- **适配 FPU 浮点/向量寄存器中断上下文保存恢复：**在 CPU 支持 FPU 和向量指令的情况下，适配中断上下文保存恢复机制以支持 FPU 寄存器和向量寄存器的正确保存和恢复，确保在中断处理过程中这些寄存器的状态能够正确维护，避免因上下文切换导致的数据损坏或异常行为。
- **离职原因：**主要是加班严重，前期无 jtag 缺少 debug 手段，最重要的只有 2 太 FPGA 机器 8-10 个人轮着用，周末基本无休都忙着 debug，导致工作效率极低，后期虽然 jtag debug 环境搭建成功了，但是项目一开始就没有考虑出 FGPA 芯片 image 时加入 JTAG 电路模块，导致很多 FGPA 芯片 image 无法使用 JTAG debug，TNPU 的 bug 也是加了 JTAG 电路模块才得以 debug 成功，严重影响了后续的开发效率和体验，最终导致自愿离职。且公司管理制度比较死板，无偿加班且不能调休，而且早上上班迟到要扣工资。经协商无果，身体难以承受长时间高强度加班，选择自愿离职。

博士视听系统(上海)有限公司 (Bose) 2024.05 - 2024.06

嵌入式软件工程师 | 上海，闵行区

- 负责可穿戴设备、蓝牙耳机等消费电子产品的上层组件开发与适配。
- **离职原因：**公司核心团队在美国，需频繁倒时差（晚上 10-11 点）开会，岗位匹配度低，自愿离职。

- **【项目一】 刚入职时 wq7033 芯片流片已完成流片，参与部分 feature 开发和 bug fix 任务。**
- **外设驱动深度适配与底层 Bug 攻坚：**主要是学习芯片架构和 riscv 指令集架构，适配新的 flash 厂商 (gega device) 的 NOR flash 驱动，期间利用逻辑分析仪抓取 SPI serial flash 协议数据包，保证初始化和读写操作符合 jdec 标准。修复过的 bug 很多，比如简单的 assert 失败到复杂的 whatch dog 超时。印象较深的一个 bug 是系统统计数据 (cpu 利用率, 任务状态等) 以及系统时间显示异常，定位到是芯片 bug，因为 RTC 的某一个寄存器是只读的，驱动认为这个应该是读写的，导致 RTC alarm 触发时间完全错乱，导致系统统计数据非常怪异。
- **FreeRTOS 低功耗与 Tickless 休眠唤醒优化：**编写 FreeRTOS 测试用例去测试验证多个驱动的正确性，包括 flash SFC (serial flash controller), UART, WDT。FreeRTOS 系统下开启 tickless IDLE 模式实测多核的低功耗 pm 模块。修复了 uart 在 Free RTOS 进出 tickless IDLE 低功耗时出现的打印乱码问题 (core1 device restore 过程中存在还没初始化 uart 就打印导致 uart1 restore 异常)，以及 wdt timer 在系统退出 tickless IDLE 恢复外设状态时无效的问题，通过把 wdt 的 save 和 restore 放到 machine mode 下作为芯片驱动的一部分而不是注册到外设驱动电源管理 pm 结构体里，这样就能抓到 pm device restore 过程中出现的死循环问题，这保证了只要系统上电就保证 wdt 有效。
- **PSRAM 控制器调优与跨子系统死锁溯源：**负责 win bound psram 厂商的 ddr psram 设备驱动支持，主要是开发调试芯片内的 SMC (Sysytem memory controller) 的驱动修复，因为当前驱动完全无法用。psram 驱动非常负责，从硬件层面 psram 设计到复杂的时钟树和电源树的正确配置，以及 bus brige 的频率切换以支持 psram 的严格时序。解决的问题，修改 dvfs 时钟电源管理代码以保证 psram 能正常上电并和 smc 通信做校准，这块涉及到深入学习理解芯片电源管理文档手册。还有一个软件无法独立解决的问题是跟多个数字部同事联合调试解决的，那就是 psram 单元测试过程中出现的读写数据出错的问题，这个是完全由数字部同事指点之下一步一步定位到是硬件默认启用的 clock gate 功能 (一直省电的技术，就是系统) 在反复换切上下电 brige 并且 频率时出现了无法消除的时钟毛刺 (无法通过调整频率和校准手段去除) 最终关闭 clock gate 功能得以解决。做了 psram 是给 hifi4 dsp 核心用的，core0 core1 dsp 核在 core0 freeRTOS 下跑低功耗经测试，发现了系统进出 tickless IDLE 后必出现的修复了非常难 debug 的 psram 数据内容被写坏且芯片挂死的 bug，经过讨论没有人知道是什么原因。独立研究调查并发现是因为给 dsp 核心上电的 pm 驱动存在 bug，给 dsp 核心上电之前没有 stall 住 dsp 核的 clock，导致一上电就开始跑，这时候 core0 core1 还没完成 psram 的初始化导致 psram 工作异常。
- **【项目二】 参与 wq7036 芯片 (7033 的升级版本，四核 riscv 核心+xtensa hifi5 dsp) 硅前验证，参与 bootrom, bootloader 开发和验证。验证了 GPIO, FLASH, ddr psram, RTC 和 gp timer, whatdog 等外设硬件功能，部分和外设相关 pmic 的验证，负责了研发一些 feature 和修补原有驱动的 bug。**
- **RISC-V PLIC 中断控制器重构与遗留架构缺陷消除：**研究 riscv PLIC 中断控制器解决了 RTC 和 gp timer 中断无法触发的问题共享中断驱动问题，一步一步排查发现 PLIC 中断控制器有正常触发中断，但是 ISR handler 没有执行，是原来的 wq7033 的 RTC 和 gp timer 存在很多硬件限制性 问题比如存在中断号共享等，7033 上的中断处理驱动有很多 work around 来修复避免的这些问题，这个 work wround 在更新的 7036 上完全失效，因为 7036 消除了中断共享以及其他硬件限制，之前的 work around 会引起 7036 的 bug
- **硅前 GPIO 验证与 PMIC 低功耗状态保持适配：**验证 gpio 功能，并配合硬件数字设计工程师 debug 和编写 gpio latch 和 PMIC 相关的低功耗模式下保持 gpio 设定功能 (上拉或下拉) 的驱动。
- **Cache/XIP 架构验证与多核共享驱动重构：**验证 cache 和 XIP, psram, cache, 修改 cache 和 sfc/smc 驱动适配新的芯片架构，消除了 7036 上多核共用 cahe 那部分 work around 驱动。

Continue of 上海物骐微电子有限公司 (Wuqi Microelectronics)

嵌入式软件工程师 | 华为项目驻场开发

- **【项目三】公司重要客户是华为和歌尔，立项 2022 年底立项 Julu2 项目（一个项目三个产品线，耳夹式蓝牙耳机，脖颈挂式骨传导蓝牙耳机，眼镜），公司安排 10-15 个人左右的团队到华为东莞松山湖公司出差，该项目主要任务是我们的 7036 芯片 SDK 适配华为的操作系统 liteOS (类 linux 系统)，适配华为的蓝牙 host 和上层应用，支持三个产品线。**
- **联合研发体系融入与 LiteOS 容器化环境构建：**学习 liteOS 系统，参与华为培训，学习如何 docker 容器化环境下嵌入式开发等等，参与华为考核，正式进入华为歌尔和我司合作的项目。
- **OpenHarmony LiteOS 深度移植与致命死锁化解：**负责 7036 的 SDK 驱动等适配到 liteOS，主要负责把 NOR flash 驱动适配到 liteOS 的 MTD device 子系统结点，gpio, RTC, gp timer, SPI, I2C, WDT 等底层驱动适配到华为 lite OS 的字符设备 char device 和 io_ctrl 系统调用 systemcall 上。主要是完善 flash 驱动的残缺和 bug，比如 liste os 要求 flash 适配写保护功能，但该功能的适配严重影响 flash 的 XIP 片内执行，导致系统异常 hang 死。深入研究 NOR flash 手册并研究 NOR flash 工作原理，发现 Nor flash 写保护功能操作 flash 内部非易失寄存器需要很长时间，这个操作会阻塞 SFC，进而阻塞 cache 和总线，cpu pipe line 也被冻住，中断不能响应，pwic 内的全局 watchdong 触发芯片复位。解决办法是，Nor flash 提供了临时改变内部控制寄存器的命令，该命令只修改 flash 内部易失控制器寄存器，这个操作立即就能成功返回，延时极短。
- **多核异构 DFx 维测基建与 Xtensa 架构上下文恢复：**负责 DFx 子系统的适配，这个也就维测子系统。针对 7036 的复杂异构系统，利用 cpu 核所在子系统下的 watchdog 和 pmic 全局 watchdog、NMI (不可屏蔽中断) 与 PMP (物理内存保护) 等硬件机制，从零构建全局异常捕获框架。精准拦截跨子系统死锁、栈溢出及非法内存访问，生成 CoreDump 文件，并用 addr2line 工具链写脚本解析系统 crashlog，快速找出系统崩溃时的源码文件行号，为这三条产品线的顺利量产提供决定性保障。该任务难点在于需要同时设计应用子系统 core0，蓝牙子系统 core1 和 xtensa hifi5 dsp 子系统的异常检查。其中，core0/core1 是 riscv 核，是我熟悉的，适配起来也容易，最难的就是 xtensa hifi5 dsp，这是个高并行执行和乱序执行的高性能 dsp，需要深刻理解 xtensa hifi5 dsp 的异常模型，通用寄存器分可见窗口寄存器 16 个，实际 register file 是 64 个，跟中断处理跟 arm Coretex-A 的 GIC 一样，分 4 个常 level el0 到 el3。
- **XIP 架构极限 JTAG 调试方法学创新：**在这个 7036 复杂异构 XIP 系统下，创造性的发现了一种利用 JTAG GDB debug 方法。之前的同事认为 JTAG GDB debug 的作用更多的在芯片硅前验证阶段，流片后几乎没人用 gdb 调试，因为 NOR flash XIP 系统上没有实现通过 cache 写入 flash 操作（难以实现，他们觉得没有必要只为了 debug 实现这块的硬件和软件），所以，没办法用 gdb load 命令去下载代码到 flash 内部，因为很多代码都是需要放到 Nor flash 里面才能执行。然后=不信邪的我非要 jtag debug，他们都叫我这是浪费时间，最终我找到了一个方法，那就既然不能 gdb load 代码，那就不用 gdb load,让 boot loader 去 load 代码，gdb 连上以后只需要打断点然后 continue。发现果然可行，如果需要 debug flash 内部的代码需要打硬件断点。这个方法也使得我能够 debug flash 卡住等艰难的问题，我还把该 debug 方法分享给团队内其他同事比如负责音频的同事，使他的音频方面的 debug 工作快速收敛并得到表彰。
- **跨子系统级核心 Bug 攻坚与千万级量产护航：**利用以上方法，为团队其他人的 debug 工作提供分析 debug 支持，解决了非常多的系统级、硬件级、跨子系统级 bug，比如 DSP 子系统异常重启，内存踩踏，内存泄漏等等。单线 uart 概率性无法通行问题，终三个产品线成功量产，并且耳夹式耳机销量火爆国内外。

湖北恒隆企业集团上海汽车电子研发有限公司 2019.04 - 2020.07

嵌入式软件工程师 | 上海，浦东新区

- 基于 NXP S32K14x 平台开发 EPS 系统存储服务模块驱动，遵循 AUTOSAR 4.x 架构与 ISO 26262 功能安全标准。
- 运用 C 语言为 AUTOSAR OS 研发模拟 EEPROM 驱动 (基于 NAND Flash)，实现高寿命磨损均衡算法以确保 20 年数据保持。
- 基于 UDS 和 CAN 协议完成存储服务模块单元测试，利用 Python 开发自动化配置文件生成工具链。
- **离职原因：**后期同事流失严重，加班急剧变多。

易兆微电子 (杭州) 有限公司 2018.03 - 2019.04

嵌入式软件工程师 | 上海, 浦东新区

- 为 ARM Cortex-M0 MCU 开发底层外设驱动 (SPI, I2C, PWM, ADC), 完成陀螺仪 (MPU-6050) 及电池监控模块驱动开发。
- 利用 Ellisys 蓝牙协议分析仪进行抓包, 诊断并解决蓝牙设备的连接丢失问题。
- 引入数学统计方法 (极大似然估计与贝叶斯公式) 处理蓝牙 RSSI 数据序列, 开发防丢设备的高精度距离估算算法。
- **离职原因:** 发展前景有限, 能学到的底层技能有限。

中国铁建重工集团有限公司新疆分公司 2017.07 - 2017.12

现场技术支持 | 乌鲁木齐

- 参与产品学习, 协助工程任务, 并在现场提供技术支持。
- **离职原因:** 专业不匹配及对工作内容缺乏兴趣, 工作 5 个月后自愿离职。

个人/开源项目 Open Source Projects

QemuRTOS

- **项目 URL:** <https://github.com/kevin-825/QemuRTOS>
- **项目简介** QemuRTOS 是为了学习研究 RTOS 内核设计而开发的实验性的实时操作系统 (RTOS) 开发平台, 目前适配 RISC-V 32/64 virt 虚拟开发板和 arm AArch64 virt 虚拟开发板和 arm Cortex-M virt 虚拟开发板。适合编写 RTOS 内核, 非常适合研究学习 GIC/APLIC/PLIC/NVIC 等硬件, 学习研究 RTOS 内核设计和实现。目前已完成 bring up 和内存管理 malloc, printf, uart ns6450 和 pl011 驱动。基于 kconfig 和 make file 的 Docker 容器化的构建系统, 支持一键编译生成可运行可执行 elf 文件和运行在 qemu 上。

QemuEmbeddedLinux

- **项目 URL:** <https://github.com/kevin-825/QemuEmbeddedLinux>
- **项目简介** QemuEmbeddedLinux 为了学习 Linux foundation 也就是 linux 基金会官方提供的 Linux kernel internal and development (LFD 420) 课程而创建的。rootfs 构建工具选择的是 buildroot, 基于 Docker 容器化的编译环境里编译。boot loader 用的 ARM ATF + uboot, 支持一键编译生成 rootfs, Image, vmlinux, uboot.bin, bl.bin, genimage 打包成 Sdcard image 和直接运行在 qemu。启用了 ccache, 编译非常快速。非常适合学习研究 Linux 内核开发, kernel module 开发, 驱动开发, 系统调优, 内核调试等。也适合学习研究 arm 和 riscv 架构的底层开发, 芯片 Bring-up 等。
- **技术栈:** C, Python, Shell Script, Buildroot, U-boot, ARM Trusted Firmware (ATF), QEMU, Docker, riscv, AARCH64.

EmbeddedDevContainer

- **项目 URL:** <https://github.com/kevin-825/EmbeddedDevContainer>
- **项目简介** EmbeddedDevContainer 是一个嵌入式开发环境容器化项目, 基于 Docker 构建, 包含了从零写的多个 Dockerfile 和 Dockerfile 模板, 适用于支持以上两个项目的容器化, 理论上经过简单修改添加内容来支持任何项目容器化开发。目前该项目包含了用于编译 GNU tool chain (gcc, gdb 等) 的 Docker image Dockerfile, 和 arm/riscv 交叉编译环境的 Docker image Dockerfile。这个项目的目标是为了让嵌入式开发者能够快速搭建一个功能完善的嵌入式开发环境, 避免繁琐的环境配置过程, 提高开发效率。
- **技术栈:** Docker, GNU tool chain, Shell Script.

专业进修 Continuous Education & Advanced Training

Linux Kernel Internals and Development (LFD420) | Linux Foundation 官方高级课程

- [Linux Foundation 官方课程主页](#)
- 深入研究 Linux 内核核心架构, 系统攻克内核模块开发、设备驱动模型、内核态高并发同步原语及性能调优。
- **核心实战:** 基于 QEMU 模拟器从零构建完整的嵌入式 Linux 系统, 独立编写并集成自定义 Kernel Module 与底层设备驱动, 并完成系统级安全加固。
- **技术栈:** C, Linux Kernel, ARM Trusted Firmware (ATF), QEMU, U-boot, Buildroot.

Developing Embedded Linux Device Drivers (LFD435) | Linux Foundation 官方高级课程

- [Linux Foundation 官方课程主页](#)
- 专注于 Linux 核心子系统与驱动框架, 深度覆盖字符设备 (Char Device)、块设备及网络设备底层实现。
- **核心实战:** 结合个人开源项目 QemuEmbeddedLinux, 完成了从系统构建、Platform Driver 到字符设备驱动开发的全生命周期演练与内核树外编译实践。
- **技术栈:** C, VFS, Device Tree (DTS), ARM Cortex-A, RISC-V.

Embedded Linux Development (LFD450) | Linux Foundation 官方高级课程

- [Linux Foundation 官方课程主页](#)
- 体系化掌握工业级嵌入式 Linux 交叉编译流程、RootFS 裁剪、性能分析定界与系统级安全架构。
- **核心实战:** 基于 RISC-V 指令集架构, 成功构建了集成 SELinux 强制访问控制 (MAC) 模块的高安全性嵌入式 Linux 基础基座。
- **技术栈:** C, Cross-compilation, SELinux, RISC-V, Buildroot.

MIT 6.004 Computation Structures | MIT OpenCourseWare (麻省理工学院公开课)

- [MIT 6.004 官方课程主页](#)
- **底层架构解构:** 深入剖析现代 CPU 微架构底层数字逻辑与硬件状态机设计。
- **核心理论基建:** 系统掌握了多级流水线 (Pipeline) 停顿与转发、MMU 与 TLB 寻址机制, 以及高级多发射乱序执行 (Out-of-Order Execution) 的核心硬件原理。
- **技术收益:** 建立起从“底层数字逻辑”到“上层操作系统”的全局硬件观, 为复杂 SoC 的时序/死锁 Debug 及内核性能优化提供了决定性的理论支撑。

教育经历 Education

2013.09 - 2017.07 **西北工业大学 (985/211、双一流)** 电子信息工程 (EE) 本科, 工学学士 西安

- 主要学习模拟/数字电路原理设计, 微机原理与架构(MCU/DSP), 信号处理 (连续/离散信号处理、统计信号处理及估计理论)

2010.09 - 2013.07 **厦门外国语学校 (厦门市重点高中)** 高中毕业 厦门

兴趣爱好 Interests

爬山, 古典瑜伽, 阅读, mysticism, 登山, yoga & meditation, 徒步旅行, 写代码, linux kernel